4.6 Sequence to sequence learning

- RNNs are used to design language models, which are key to natural language processing
- another important benchmark is *machine translation*, a central problem domain for *sequence transduction* models that transform input sequences into output sequences
- playing a crucial role in various modern AI applications, sequence transduction models will form the focus of the remainder of this chapter and Chapter 5
- thus, we introduce the machine translation problem

4.6 Sequence to sequence learning

- *machine translation* refers to the automatic translation of a sequence from one language to another
- for decades, statistical approaches had been dominant in this field, before the rise of end-to-end learning using neural networks
- the latter is often called *neural machine translation* to distinguish itself from *statistical machine translation* that involves statistical analysis in components such as the translation model and the language model
- we will focus on neural machine translation methods
- different from language models, for which the corpus is in one single language, machine translation datasets are composed of pairs of text sequences that are in the *source language* and the *target language*, respectively

4.6 Sequence to sequence learning

- thus, machine translation is a major problem domain for sequence transduction models, whose input and output are both variable-length sequences
- to handle this type of inputs and outputs, we can design an architecture with two major components
- the first component is an *encoder*: it takes a variable-length sequence as the input and transforms it into a state with a fixed shape
- the second component is a *decoder*: it maps the encoded state of a fixed shape to a variable-length sequence
- this is called an *encoder-decoder* architecture, which is depicted in Figure 13

4.6 Sequence to sequence learning

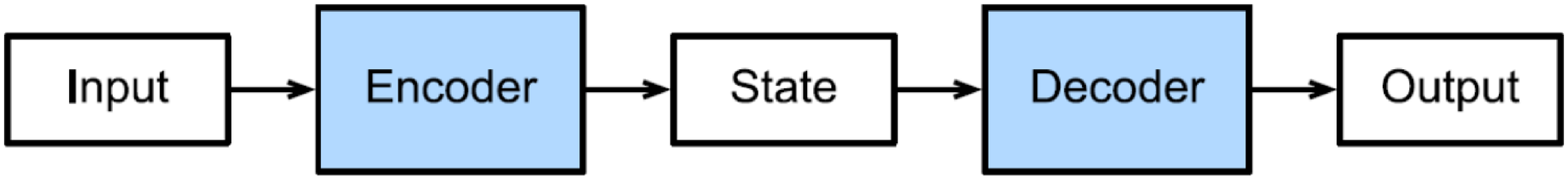


Figure 13: The encoder-decoder architecture.

4.6 Sequence to sequence learning

- let us take machine translation from English to French as an example
- given an input sequence in English: “They”, “are”, “watching”, “.”, this encoder-decoder architecture first encodes the variable-length input into a state, then decodes the state to generate the translated sequence token by token as the output: “Ils”, “regardent”, “.”
- the term “state” in the encoder-decoder architecture has probably inspired us to think about neural networks with states, when instantiating this architecture
- next, we will see how to apply RNNs to design sequence transduction models based on this encoder-decoder architecture

4.6 Sequence to sequence learning

- in machine translation, both the input and output are a variable-length sequence
- to address this type of problem, we have designed a general encoder-decoder architecture
- now, we will use two RNNs to design the encoder and the decoder of this architecture, and apply it to sequence to sequence learning for machine translation

4.6 Sequence to sequence learning

- following the design principle of the encoder-decoder architecture, the RNN encoder can take a variable-length sequence as the input, and transform it into a fixed-shape hidden state
- in other words, information of the input (source) sequence is *encoded* in the hidden state of the RNN encoder
- to generate the output sequence token by token, a separate RNN decoder can predict the next token based on what tokens have been seen (such as in language modeling) or generated, together with the encoded information of the input sequence
- Figure 14 illustrates how to use two RNNs for sequence to sequence learning in machine translation

4.6 Sequence to sequence learning

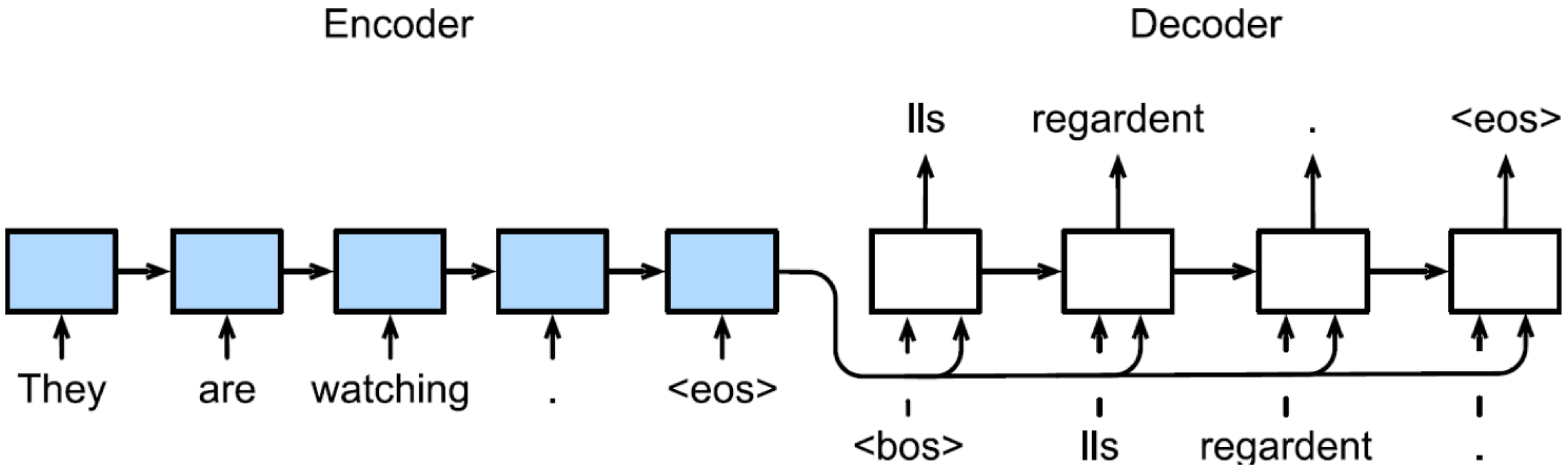


Figure 14: Sequence to sequence learning with an RNN encoder and an RNN decoder.

4.6 Sequence to sequence learning

- in Figure 14, the special “<eos>” token marks the end of the sequence
- the model can stop making predictions once this token is generated
- at the initial time step of the RNN decoder, there are two special design decisions
- first, the special beginning-of-sequence “<bos>” token is an input
- second, the final hidden state of the RNN encoder is used to initialize the hidden state of the decoder

4.6 Sequence to sequence learning

- in some designs, this is exactly how the encoded input sequence information is fed into the decoder for generating the output (target) sequence
- in some other designs, the final hidden state of the encoder is also fed into the decoder as part of the inputs at every time step, as shown in Figure 14
- similar to the training of language models, we can allow the labels to be the original output sequence, shifted by one token: “<bos>”, “Ils”, “regardent”, “.” $\rightarrow$ “Ils”, “regardent”, “.”, “<eos>”
- in the following, we will explain the design of Figure 14 in greater detail

4.6 Sequence to sequence learning

- technically speaking, the encoder transforms an input sequence of variable length into a fixed-shape *context variable* $\boldsymbol{c}$, and encodes the input sequence information in this context variable
- as depicted in Figure 14, we can use an RNN to design the encoder
- let us consider a sequence example (batch size: 1)
- suppose that the input sequence is $x_1, \ldots, x_T$, such that x_t is the tth token in the input text sequence
- at time step t, the RNN transforms the input feature vector $\boldsymbol{x}_t$ for x_t and the hidden state $\boldsymbol{h}_{t-1}$ from the previous time step into the current hidden state $\boldsymbol{h}_t$
- we can use a function f to express the transformation of the RNN's recurrent layer:

$$\boldsymbol{h}_t = f(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}).$$

4.6 Sequence to sequence learning

- in general, the encoder transforms the hidden states at all the time steps into the context variable through a customized function q:

$$\boldsymbol{c} = q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T).$$

- for example, when choosing $q(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_T) = \boldsymbol{h}_T$, such as in Figure 14, the context variable is just the hidden state $\boldsymbol{h}_T$ of the input sequence at the final time step
- so far, we have used an RNN to design the encoder, where a hidden state only depends on the input subsequence at, and before, the time step of the hidden state

4.6 Sequence to sequence learning

- as we just mentioned, the context variable $\boldsymbol{c}$ of the encoder's output encodes the entire input sequence $x_1, \ldots, x_T$
- given the output sequence $y_1, y_2, \ldots, y_{T'}$ from the training dataset, for each time step t' (the symbol differs from the time step t of input sequences or encoders), the probability of the decoder output $y_{t'}$ is conditional on the previous output subsequence $y_1, \ldots, y_{t'-1}$, and the context variable $\boldsymbol{c}$, i.e., $P(y_{t'}|y_1, \ldots, y_{t'-1}, \boldsymbol{c})$

4.6 Sequence to sequence learning

- to model this conditional probability on sequences, we can use another RNN as the decoder
- at any time step t' on the output sequence, the RNN takes the output $y_{t'-1}$ from the previous time step and the context variable $\boldsymbol{c}$ as its input, then transforms them and the previous hidden state $\boldsymbol{s}_{t'-1}$ into the hidden state $\boldsymbol{s}_{t'}$ at the current time step
- as a result, we can use a function g to express the transformation of the decoder's hidden layer:

$$\boldsymbol{s}_{t'} = g(y_{t'-1}, \boldsymbol{c}, \boldsymbol{s}_{t'-1}).$$

4.6 Sequence to sequence learning

- after obtaining the hidden state of the decoder, we can use an output layer and the softmax operation to compute the conditional probability distribution $P(y_{t'} | y_1, \ldots, y_{t'-1}, \boldsymbol{c})$ for the output at time step t'
- to predict the output sequence token by token, at each decoder time step, the predicted token from the previous time step is fed into the decoder as an input
- similar to training, at the initial time step, the beginning-of-sequence (“<bos>”) token is fed into the decoder
- this prediction process is illustrated in Figure 15
- when the end-of-sequence (“<eos>”) token is predicted, the prediction of the output sequence is complete

4.6 Sequence to sequence learning

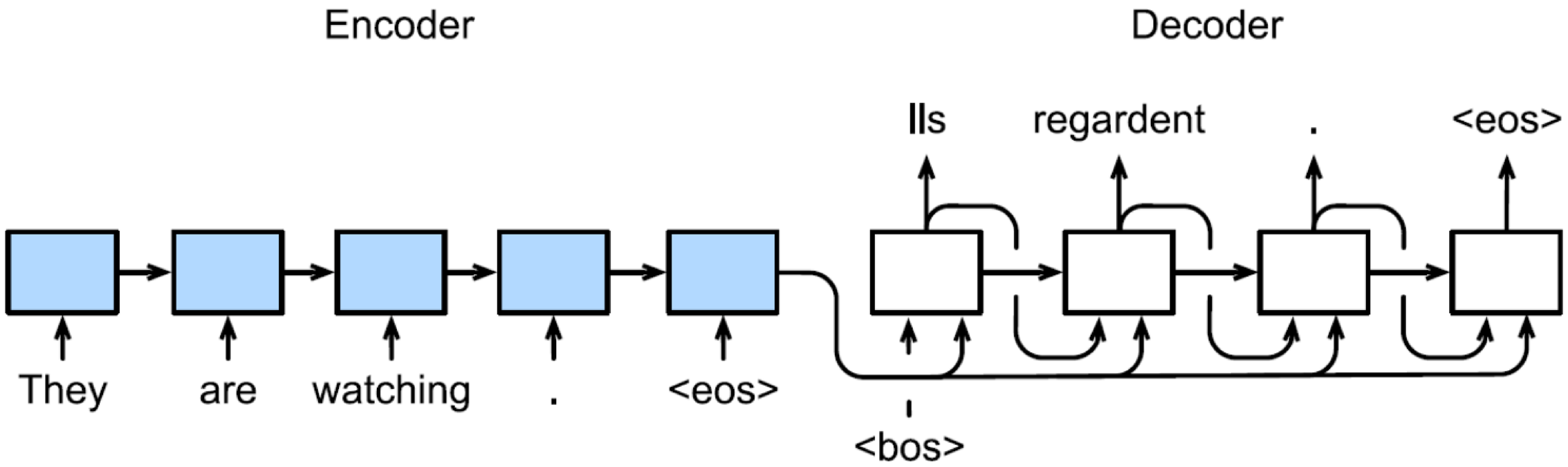


Figure 15: Predicting the output sequence token by token using an RNN encoder-decoder.

4.6 Sequence to sequence learning

- we can evaluate a predicted sequence by comparing it with the label sequence (the ground truth)
- *BLEU* (*Bilingual Evaluation Understudy*), though originally proposed for evaluating machine translation results, has been extensively used in measuring the quality of output sequences for different applications
- in principle, for any n-grams in the predicted sequence, BLEU evaluates whether these n-grams appear in the label sequence

4.6 Sequence to sequence learning

- denote by p_n the precision of n-grams, which is the ratio of the number of matched n-grams in the predicted and label sequences to the number of n-grams in the predicted sequence
- to explain, given a label sequence A, B, C, D, E, F, and a predicted sequence A, B, B, C, D, we have $p_1 = 4/5$, $p_2 = 3/4$, $p_3 = 1/3$, and $p_4 = 0$
- besides, let $\text{len}_{\text{label}}$ and len_{pred} be the number of tokens in the label sequence and the predicted sequence, respectively
- then, BLEU is defined as:

$$\exp\left(\min\left(0, 1 - \frac{\text{len}_{\text{label}}}{\text{len}_{\text{pred}}}\right)\right) \prod_{n=1}^{k} p_n^{1/2^n}, \tag{7}$$

where k is the longest n-grams for matching

4.6 Sequence to sequence learning

- based on the definition of BLEU in (7), whenever the predicted sequence is the same as the label sequence, BLEU is 1
- moreover, since matching longer n-grams is more difficult, BLEU assigns a greater weight to a longer n-gram precision
- specifically, when p_n is fixed, $p_n^{1/2^n}$ increases as n grows (the original paper uses $p_n^{1/n}$)
- furthermore, since predicting shorter sequences tends to obtain a higher p_n value, the coefficient before the multiplication term in (7) penalizes shorter predicted sequences
- for example, when $k = 2$, given the label sequence A, B, C, D, E, F and the predicted sequence A, B, although $p_1 = p_2 = 1$, the penalty factor $\exp(1 - 6/2) \approx 0.14$ lowers the BLEU